

Access to everything. Now up to 60% off. [Upgrade now](#)

Part 1: `std::inplace_vector` — The Fixed-Capacity Vector Finally Comes to C++26

Learn what `std::inplace_vector` is, how it works, and when to choose it over `std::vector` and `std::array`.

9 min read · 4 hours ago



Sagar

Following ▾

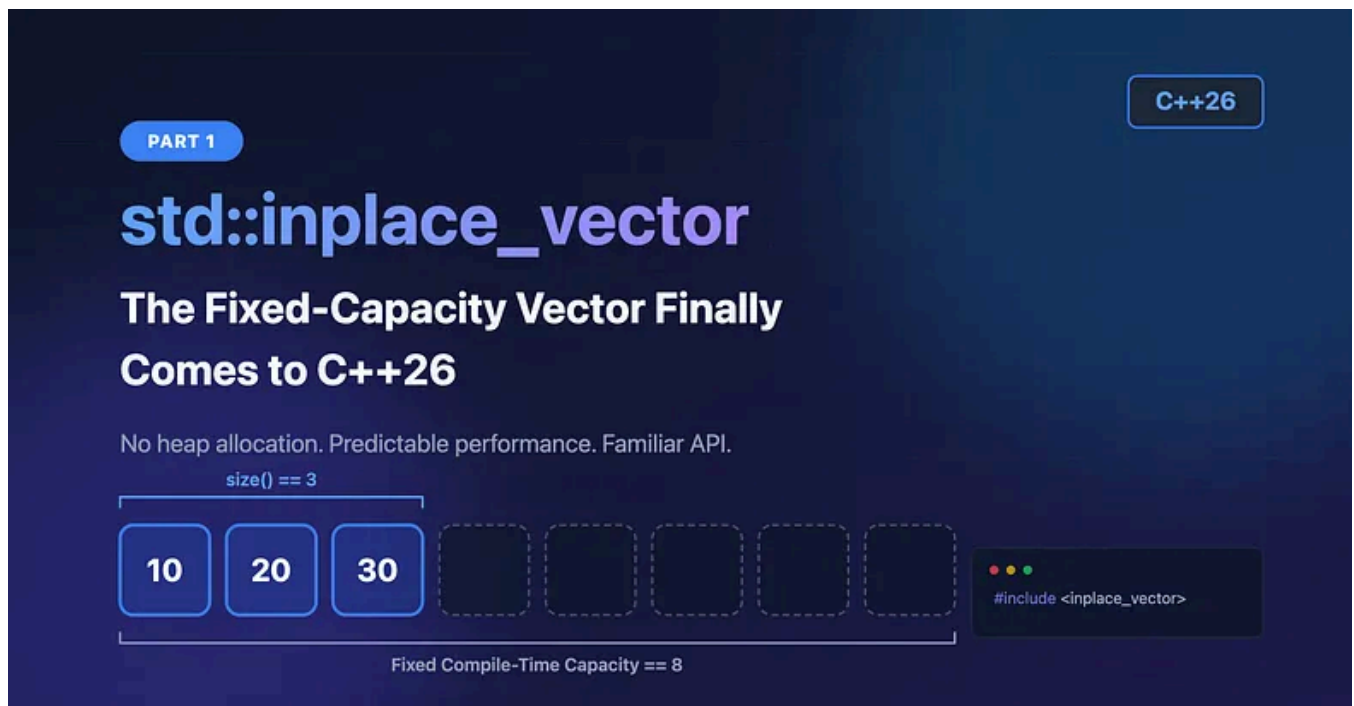


Listen



Share

⋮ More



There are very few additions to the C++ standard library that immediately make you think, “*I’ve written this container before.*” `std::inplace_vector` is one of them.

For years, projects that cared about predictable performance have relied on their own fixed-capacity vector implementations. Boost has `static_vector`. EASTL has

one. Folly has one. Plenty of game engines, embedded systems, trading platforms, and networking libraries have their own version as well.

C++26 finally standardizes the idea.

At first glance, `std::inplace_vector` looks like a `std::vector` with a compile-time capacity. That's true, but it doesn't really explain why the committee felt it deserved its own container instead of being another optimization of `std::vector`.

The real story is much more interesting.

The feature was introduced through [P0843R12](#), a proposal that spent years evolving within WG21. Beyond defining the API, the paper captures the reasoning behind many of the design decisions — from exception handling and move semantics to why the container intentionally mirrors `std::vector`. Throughout this article, we'll not only look at how `std::inplace_vector` works, but also explore some of the thinking that shaped its final design.

. . .

What is wrong with our good Old friend `std::vector`?

`std::vector` is an excellent default container. It grows automatically, offers contiguous storage, works well with algorithms, and has decades of optimization behind it.

The catch is that its storage lives on the heap.

```
std::vector<int> values;

values.push_back(1);
values.push_back(2);
values.push_back(3);
```

Nothing unusual here. The vector allocates memory as needed, grows when its capacity is exhausted, and frees that memory when it is destroyed. Most of the time that's exactly what you want.

Sometimes it isn't. Imagine you're

- processing packets inside a networking stack. Every packet can contain at most eight headers.
- Or maybe you're writing a compiler where an AST node never has more than four child nodes.
- Or perhaps you're working on firmware where dynamic allocation isn't even available.

In all of these situations, you already know the maximum number of elements. Paying for dynamic allocation doesn't buy you anything.

So before C++26 we came up with workarounds.

The Common Workarounds

Most developers end up choosing one of three options.

The first is simply using `std::vector` anyway.

```
std::vector<Token> tokens;  
tokens.reserve(16);
```

Reserving capacity certainly helps reduce reallocations, but the initial allocation still comes from the heap. The memory also exists independently of the vector object itself.

The second option is `std::array`.

```
std::array<Token, 16> tokens;
```

This avoids allocation entirely. Unfortunately, it creates sixteen `Token` objects immediately. That becomes a problem when:

- `Token` isn't default constructible
- constructing every element is expensive
- you only intend to use a handful of them

`std::array` models a fixed-size collection.

*Sometimes you need a collection whose **capacity** is fixed while its **size** changes during execution. Those are two different problems.*

The third option is writing a **custom container**.

If you've spent enough time in modern C++, you've probably seen something resembling this:

```
template<typename T, std::size_t Capacity>
class StaticVector
{
    alignas(T) std::byte storage[sizeof(T) * Capacity];
    std::size_t size = 0;

    // ...
};
```

Inside you'll usually find: placement new, explicit destructor calls, manual lifetime management, pointer arithmetic, exception handling, dozens of edge cases. It works. Until somebody discovers a bug in the move constructor six months later.

Neither `std::array` nor `std::vector` is a perfect fit for this use case.

C++26 addresses that gap with `std::inplace_vector`, a container that combines the fixed-capacity storage of `std::array` with much of the familiar interface and flexibility of `std::vector`.

Feature	<code>std::array</code>	<code>std::vector</code>
Runtime size	Fixed	Dynamic
Maximum capacity	Fixed	Dynamic
Heap allocation	No	Yes
Default-constructs all elements	Yes	No
Supports non-default-constructible types	Limited	Yes
Storage	Inline	Heap

• • •

C++26 `std::inplace_vector`

`std::inplace_vector` standardizes this pattern. Its declaration is refreshingly simple.

```
#include <inplace_vector>

std::inplace_vector<int, 8> values;
```

The second template argument is the important one. `8` isn't the initial size. It isn't a reserve hint. It isn't a growth factor. It is the **maximum number of elements the container will ever hold**. Unlike `std::vector`, this capacity never changes.

Using it feels almost identical to using a normal vector.

```
std::inplace_vector<std::string, 4> names;

names.push_back("Alice");
names.push_back("Bob");
names.emplace_back("Charlie");
```

The familiar API is intentional.

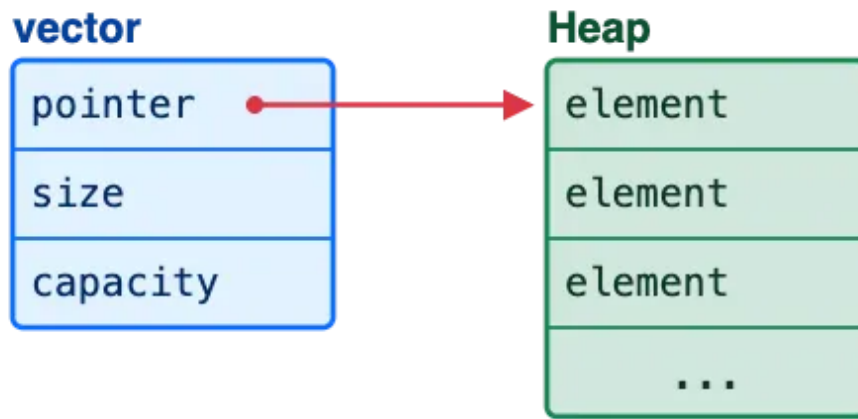
One of the design goals of the proposal was to minimize the learning curve. If you already know `std::vector`, almost everything carries over. That familiarity is one of the nicest aspects of the design. You don't have to learn a completely different container. You mostly just stop thinking about allocation.

• • •

The Biggest Difference Isn't the API

The real difference is where the elements live.

With a normal vector, the object itself is tiny. The vector merely owns memory somewhere else.



`std::inplace_vector` turns this inside out.



The storage is embedded directly inside the object.

No separate allocation exists.

Moving the container means moving the elements themselves because there isn't a pointer that can simply be transferred.

This seemingly small implementation detail explains many of the design decisions in the proposal.

- It explains why move operations are linear.

- It explains why swap is linear.
- It explains why iterator invalidation differs from `std::vector`.
- It even explains why allocator support was deliberately left out.

Almost everything follows from this one design choice.

. . .

Size and Capacity Are No Longer Related

With `std::vector`, capacity changes over time.

```
std::vector<int> numbers;  
  
numbers.reserve(64);
```

With `std::inplace_vector`, capacity is fixed forever.

```
std::inplace_vector<int, 64> numbers;
```

The container still has a runtime size.

```
numbers.push_back(1);  
numbers.push_back(2);  
  
std::cout << numbers.size();    // 2  
std::cout << numbers.capacity(); // 64
```

The important difference is that `capacity()` is known when the type is instantiated. It never grows. It never shrinks. Every instance of `std::inplace_vector<int, 64>` reserves space for exactly sixty-four integers inside itself.

Does This Mean Everything Is Constructed Up Front?

No. This is one of the easiest misconceptions to have. The storage exists. The objects do not. When you create:

```
std::inplace_vector<std::string, 32> names;
```

you are **not** constructing thirty-two `std::string` objects.



Only the storage is reserved. Each `std::string` is constructed when you insert it and destroyed when you erase it.

This is exactly why `std::inplace_vector` can store types that are not default constructible —something `std::array` cannot do.

That distinction is subtle, but it's fundamental to how the container works.

. . .

The API Feels Familiar — Until It Doesn't

One of the nicest things about `std::inplace_vector` is that you don't need to learn an entirely new container. Most of the interface mirrors `std::vector`.

```
std::inplace_vector<int, 8> numbers;

numbers.push_back(1);
numbers.emplace_back(2);

numbers.insert(numbers.begin(), 0);

numbers.erase(numbers.begin());
```



```
numbers.pop_back();
```

Iteration is exactly what you'd expect.

```
for (auto& value : numbers)
{
    std::cout << value << ' ';
}
```

It also supports contiguous storage, so `data()` works just like `std::vector`.

```
process(numbers.data(), numbers.size());
```

If an API already accepts a `std::vector`, chances are it'll work just as well with an `std::inplace_vector`.

That familiarity wasn't accidental. One of the primary goals of the proposal was to make switching between the two containers as painless as possible.

But once you reach the container's capacity, things become interesting.

. . .

What Happens When the Container Is Full?

Imagine this.

```
std::inplace_vector<int, 3> numbers;

numbers.push_back(10);
numbers.push_back(20);
numbers.push_back(30);

numbers.push_back(40);
```

The last insertion cannot succeed.

There isn't any memory left.

Unlike `std::vector`, there is nowhere to allocate additional storage.

So what should happen?

The obvious answer seems to be throwing `std::length_error`. After all, the container has reached its maximum size. That wasn't the committee's choice.

Instead, `std::inplace_vector` throws `std::bad_alloc`.

At first, that sounds strange. Nothing was allocated. No allocator was even involved.

. . .

So Why `std::bad_alloc`?

This decision generated a surprising amount of discussion. The reasoning wasn't about technical correctness. It was about making existing code easier to adapt.

Imagine a large codebase where `std::vector` is replaced with `std::inplace_vector` as a performance optimization.

Existing code may already handle allocation failures like this.

```
try
{
    values.push_back(item);
}
catch (const std::bad_alloc&)
{
    recover();
}
```

If `std::inplace_vector` threw a completely different exception, that code would silently stop handling failures correctly. By continuing to throw `std::bad_alloc`, switching between the two containers becomes much safer.

It's one of those decisions that's less about theory and more about practical software engineering.

• • •

But Sometimes You Don't Want Exceptions

Not every project relies on exceptions. Embedded software is a good example. For those situations, C++26 adds something that `std::vector` has never had.

```
try_push_back()
```

Instead of throwing, it simply reports whether insertion succeeded.

```
std::inplace_vector<int, 2> values;

values.try_push_back(1);
values.try_push_back(2);

if (!values.try_push_back(3))
{
    std::cout << "Container is full\n";
}
```

Notice the return type. It doesn't return `bool`. It returns a pointer to the inserted element.

```
if (auto* value = values.try_push_back(42))
{
    std::cout << *value;
}
```

Returning a pointer avoids an extra lookup while still making failure easy to detect with `nullptr`.

Another benefit is that, if insertion fails, an rvalue argument isn't consumed.

```
Widget widget;

if (!widgets.try_push_back(std::move(widget)))
{
    // widget has NOT been moved from
}
```

That's a surprisingly useful guarantee when retrying an operation somewhere else.

• • •

Then There's `unchecked_push_back`

If checking capacity every time is unnecessary, C++26 also provides an unchecked alternative.

```
values.unchecked_push_back(42);
```

There's no exception. There's no return value indicating failure. There isn't even a capacity check. The only requirement is simple.

```
size() < capacity()
```

If that isn't true, the program has **undefined behavior**. This isn't a design mistake. It's deliberate. Imagine you're filling an `inplace_vector` after validating the input size once.

```
assert(count <= values.capacity());

for (...)
{
    values.unchecked_push_back(...);
}
```

Performing the same bounds check on every iteration accomplishes nothing except adding another branch. The unchecked API exists specifically for situations like this.

It's a small optimization, but exactly the kind of optimization developers using this container tend to care about.

. . .

`try_emplace_back` **Works Exactly the Way You Expect**

Just like `emplace_back`, objects are constructed directly inside the container.

```
std::inplace_vector<std::string, 4> names;  
  
names.try_emplace_back(5, 'A');
```

This creates `AAAAA` without first constructing a temporary `std::string`.

If the container is already full, nothing happens and `nullptr` is returned. Again, no exceptions. No partially moved objects. No surprises.

. . .

Range Operations Are Included Too

Modern C++ containers increasingly support ranges, and `std::inplace_vector` follows suit. Appending an entire range is straightforward.

```
std::array values = {1, 2, 3};  
std::inplace_vector<int, 8> numbers;  
numbers.append_range(values);
```

There is also a fallible version.

```
auto it = numbers.try_append_range(values);

if (it != values.end())
{
    std::cout << "Not everything fit.\n";
}
```

Instead of throwing, it returns an iterator to the first element that couldn't be inserted. This is especially useful when processing streams of data where partial progress is acceptable.

. . .

Reserve Doesn't Really Reserve Anything

One function looks a little odd.

```
numbers.reserve(16);
```

Normally, `reserve()` allocates additional storage. Here, that's impossible. The capacity was fixed when the type was instantiated.

So why does the function exist?

Mainly for compatibility. Generic code often calls `reserve()` without caring which vector-like container it's working with.

With `std::inplace_vector`, calling `numbers.reserve(8)` does absolutely nothing.

Calling `numbers.reserve(100)` throws `std::bad_alloc` because the requested capacity can never be satisfied.

Similarly `numbers.shrink_to_fit()` is simply a no-op. There's nothing to shrink. The storage is already embedded inside the object.

. . .

Why There Are No Deduction Guides?

One thing you might expect to work is this.

```
std::inplace_vector numbers = {1, 2, 3};
```

It doesn't. Unlike `std::vector`, template argument deduction isn't possible here. The compiler can easily deduce the element type `int`.

But what should it choose for the capacity?

Three? Eight? Thirty-two? One hundred?

There isn't a universally correct answer.

Rather than inventing rules that would surprise people, the standard simply requires the capacity to be specified explicitly.

```
std::inplace_vector<int, 8> numbers = {1, 2, 3};
```

It's a small inconvenience, but probably the least surprising design.

• • •

Closing Thoughts

By now, `std::inplace_vector` probably looks like exactly what its name suggests—a fixed-capacity `std::vector`.

Here's a quick summary of where each container fits:

Container	Key Characteristics
<code>std::vector</code>	Heap-allocated storage, dynamic capacity, amortized growth, constant-time move
<code>std::array</code>	Inline storage, fixed size, constructs every element up front
<code>std::inplace_vector</code>	Inline storage, fixed capacity with runtime size, lazy element construction, linear-time move

Under the hood, though, its implementation is very different. Those differences show up in places that aren't immediately obvious, particularly move operations,

iterator invalidation, object lifetime, and trivial copyability.

Those design decisions are where the proposal becomes especially interesting, and we'll dive into them in the **next part**.

• • •

Found this article helpful? Please **Clap** 🙌 and **follow** for more C++ and system programming content.

Programming

Technology

Cpp26

C Plus Plus Language

Software Development



Following ▾

Written by Sagar

224 followers · 7 following

System Programmer | C++ | Linux System Programming | Rust | Python | Cloud Computing | DevOps | Low Latency Backends



Placeholder for a bio or short description.

Placeholder for a list of articles or topics.

Placeholder for a link or tag.

Placeholder for a link or tag.

